Lab exercise

Francisco Azeredo, PhD

Feedforward Neural Networks

Learning Objectives

Students will develop a clear understanding of how a feedforward neural network operates by computing each step of the forward pass manually and then verifying the same computations using PyTorch. Through this process, they will learn how inputs are transformed across layers, how weights and biases shape these transformations, and how activation functions introduce nonlinearity. The lab emphasizes careful tracking of dimensions, reinforcing how data flows through the network, and helps students connect the mathematical structure of neural networks to their implementation in modern deep learning frameworks.

Important

All manual computations must be shown. Answers without intermediate steps will not receive full credit.

Task

0. Using the input vector X and the parameters of the first layer, compute the first step of the forward pass manually. For $l = 1$,

$$h_1 = W^{(1)}X + b^{(1)}, \qquad \sigma_1 = \sigma^{(1)}(h_1),$$

with $\sigma^{(1)}(h_1) = \max(0, h_1)$ applied element-wise.

The input is

$$X = \begin{bmatrix} 1.0 \\ -2.0 \\ 3.0 \end{bmatrix}.$$

The parameters are

$$W^{(1)} = \begin{bmatrix} 0.2 & -0.5 & 1.0 \\ 1.5 & 0.3 & -0.8 \\ -1.0 & 0.7 & 0.2 \\ 0.1 & -0.2 & 0.3 \end{bmatrix}, \qquad b^{(1)} = \begin{bmatrix} 0.5 \\ -0.1 \\ 0.0 \\ 0.2 \end{bmatrix}.$$

Compute the vector $h_1 \in \mathbb{R}^{4\times 1}$ and then apply ReLU to obtain σ_1. Report h_1 and σ_1. Then print the dimensions of X, $W^{(1)}$, $b^{(1)}$, h_1, and σ_1. Show your computations clearly by computing each component of h_1 explicitly, including the full dot product for at least one node. Briefly explain, step by step, how the transformation from X to σ_1 reflects the mapping from the input features to the nodes of the first hidden layer.

1. Using the vector σ_1 from the previous step and the parameters of the second layer, compute the next step of the forward pass manually. For $l = 2$,

$$h_2 = W^{(2)}\sigma_1 + b^{(2)}, \qquad \sigma_2 = \sigma^{(2)}(h_2),$$

with $\sigma^{(2)}(h_2) = \max(0, h_2)$ applied element-wise.

The input to this layer is

$$\sigma_1 \in \mathbb{R}^{4\times 1}.$$

The parameters are

$$W^{(2)} = \begin{bmatrix} 0.3 & -1.2 & 0.5 & 0.7 \\ -0.6 & 0.8 & -0.3 & 0.9 \end{bmatrix}, \qquad b^{(2)} = \begin{bmatrix} 0.0 \\ 0.1 \end{bmatrix}.$$

Compute the vector $h_2 \in \mathbb{R}^{2\times 1}$ and then apply ReLU to obtain σ_2. Report h_2 and σ_2. Then print the dimensions of σ_1, $W^{(2)}$, $b^{(2)}$, h_2, and σ_2. Show your computations clearly by computing each component of h_2 explicitly, including the full dot product for at least one node. Briefly explain, step by step, how the transformation from σ_1 to σ_2 reflects the mapping from the first hidden layer to the nodes of the second hidden layer.

2. Using the vector σ_2 from the previous step and the parameters of the output layer, compute the final step of the forward pass manually. For $l = 3$,

$$h_3 = W^{(3)}\sigma_2 + b^{(3)}, \qquad \sigma_3 = \sigma^{(3)}(h_3),$$

where the activation function is the identity:

$$\sigma^{(3)}(h_3) = h_3.$$

The input to this layer is

$$\sigma_2 \in \mathbb{R}^{2\times 1}.$$

The parameters are

$$W^{(3)} = \begin{bmatrix} 1.0 & -1.5 \end{bmatrix}, \qquad b^{(3)} = \begin{bmatrix} 0.2 \end{bmatrix}.$$

Compute the scalar $h_3 \in \mathbb{R}^{1\times 1}$ and then apply the identity function to obtain σ_3. Report h_3 and σ_3. Then print the dimensions of σ_2, $W^{(3)}$, $b^{(3)}$, h_3, and σ_3. Show your computations clearly. Briefly explain how the transformation from σ_2 to σ_3 produces the final output of the network.

3. Using the same input X and parameters $\{W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}, W^{(3)}, b^{(3)}\}$, construct a feedforward neural network in PyTorch that reproduces the same computations.

   Define a model using `torch.nn` with:

   - a first linear layer mapping $\mathbb{R}^3 \to \mathbb{R}^4$,
   - a second linear layer mapping $\mathbb{R}^4 \to \mathbb{R}^2$,
   - a final linear layer mapping $\mathbb{R}^2 \to \mathbb{R}^1$,
   - ReLU activations for the first two layers and identity for the output.

   Ensure that the input is correctly formatted as a row vector (batch dimension first). Load the predefined weights and biases into the model, perform a forward pass, and report the intermediate outputs.

   Compare each manually computed pair (h_l, σ_l) with the corresponding PyTorch outputs and verify that they match in value and dimension (after aligning orientation). Briefly explain any differences in tensor orientation between the manual and PyTorch computations.

4. Compute manually the total number of parameters in the network. Report the total number of parameters. Show how many parameters come from each weight matrix and bias vector. Then, verify your result using PyTorch. Briefly explain how the number of parameters depends on the layer dimensions.

5. Modify the network so that the output layer produces a probability vector over 3 classes using the softmax function. Specifically, replace the final layer with:

   $$h_3 = W^{(3)}\sigma_2 + b^{(3)}, \qquad \sigma_3 = \mathrm{softmax}(h_3),$$

   where

   $$\mathrm{softmax}(h_3)_i = \frac{e^{h_{3,i}}}{\sum_{j=1}^{3} e^{h_{3,j}}}, \quad i = 1, 2, 3,$$

   and $W^{(3)} \in \mathbb{R}^{3\times 2}$, $b^{(3)} \in \mathbb{R}^{3\times 1}$.

Redefine $W^{(3)}$ and $b^{(3)}$ accordingly, and repeat Tasks 2–4 for this modified network. Report the final output σ_3 and verify that its entries are non-negative and sum to 1. Compare your manually computed σ_3 with the PyTorch output σ_3 and confirm that they match. Explain why softmax is appropriate for multi-class classification and how the interpretation of the output differs from the scalar output in Task 2.